

Relatorio do trial

PARTICIPANTE

Luis Henrique Gonçalves Barbosa

KATA

Tarifador de Estacionamento Rotativo

TRATAMENTO

Com assistente de IA

TEMPO FINAL

01:14

TEMPO ESGOTADO

Nao

INICIO

16/09/2026, 13:00:00

FIM

16/09/2026, 13:01:14

RESULTADO E OBSERVACOES

8 de 8 testes unitários de aceitação passaram com sucesso (100%).

ARQUIVOS ANEXADOS

3 arquivo(s) de código ou texto anexado(s) ao trial.

- TarifadorEstacionamento.java (69 linhas)
- TipoVeiculo.java (18 linhas)
- texto-do-trial.txt (69 linhas)

TarifadorEstacionamento.java

```
package br.ufc.lab02.estacionamento;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.LocalDateTime;

public class TarifadorEstacionamento {

    private static final BigDecimal TARIFA_PRIMEIRA_HORA = BigDecimal.valueOf(12.00);
    private static final BigDecimal TARIFA_HORA_ADICIONAL = BigDecimal.valueOf(6.00);
    private static final BigDecimal TETO_DIARIA = BigDecimal.valueOf(60.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.85);

    public BigDecimal calcularTarifa(LocalDateTime entrada, LocalDateTime saida,
        TipoVeiculo tipo, boolean possuiConvenio) {
        if (entrada == null || saida == null) {
            throw new IllegalArgumentException("Entrada e saida nao podem ser nulas.");
        }
        if (saida.isBefore(entrada)) {
            throw new IllegalArgumentException("Saida nao pode ser anterior a entrada.");
        }
        if (tipo == null) {
            throw new IllegalArgumentException("Tipo de veiculo obrigatorio.");
        }

        long minutosTotais = Duration.between(entrada, saida).toMinutes();

        // 1. Franquia de 15 minutos
        if (minutosTotais <= 15) {
            return BigDecimal.ZERO.setScale(2, RoundingMode.HALF_EVEN);
        }

        // 2. Cálculo do total de dias completos e minutos restantes
        long diasCompleto = minutosTotais / (24 * 60);
        long minutosRestantes = minutosTotais % (24 * 60);

        BigDecimal valorBase = TETO_DIARIA.multiply(BigDecimal.valueOf(diasCompleto));

        if (minutosRestantes > 0) {
            BigDecimal valorPeriodoRestante;
            if (minutosRestantes <= 60) {
                valorPeriodoRestante = TARIFA_PRIMEIRA_HORA;
            } else {
                long minutosAposPrimeiraHora = minutosRestantes - 60;
                long horasAdicionais = (minutosAposPrimeiraHora + 59) / 60;
                valorPeriodoRestante = TARIFA_PRIMEIRA_HORA.add(
                    TARIFA_HORA_ADICIONAL.multiply(BigDecimal.valueOf(horasAdicionais))
                );
            }

            // Aplica teto da diária sobre o período restante
            if (valorPeriodoRestante.compareTo(TETO_DIARIA) > 0) {
                valorPeriodoRestante = TETO_DIARIA;
            }
            valorBase = valorBase.add(valorPeriodoRestante);
        }
    }
}
```

```
        // 3. Multiplicador pelo tipo de veículo
        BigDecimal valorComVeiculo =
valorBase.multiply(BigDecimal.valueOf(tipo.getFatorMultiplicador()));

        // 4. Desconto de convênio
        BigDecimal valorFinal = possuiConvenio
            ? valorComVeiculo.multiply(FATOR_DESCONTO_CONVENIO)
            : valorComVeiculo;

        return valorFinal.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

TipoVeiculo.java

```
package br.ufc.lab02.estacionamento;

public enum TipoVeiculo {
    MOTO(0.50),
    CARRO_PASSEIO(1.00),
    CAMINHONETE(1.25);

    private final double fatorMultiplicador;

    TipoVeiculo(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}
```

texto-do-trial.txt

```
package br.ufc.lab02.estacionamento;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.LocalDateTime;

public class TarifadorEstacionamento {

    private static final BigDecimal TARIFA_PRIMEIRA_HORA = BigDecimal.valueOf(12.00);
    private static final BigDecimal TARIFA_HORA_ADICIONAL = BigDecimal.valueOf(6.00);
    private static final BigDecimal TETO_DIARIA = BigDecimal.valueOf(60.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.85);

    public BigDecimal calcularTarifa(LocalDateTime entrada, LocalDateTime saida,
        TipoVeiculo tipo, boolean possuiConvenio) {
        if (entrada == null || saida == null) {
            throw new IllegalArgumentException("Entrada e saida nao podem ser nulas.");
        }
        if (saida.isBefore(entrada)) {
            throw new IllegalArgumentException("Saida nao pode ser anterior a entrada.");
        }
        if (tipo == null) {
            throw new IllegalArgumentException("Tipo de veiculo obrigatorio.");
        }

        long minutosTotais = Duration.between(entrada, saida).toMinutes();

        // 1. Franquia de 15 minutos
        if (minutosTotais <= 15) {
            return BigDecimal.ZERO.setScale(2, RoundingMode.HALF_EVEN);
        }

        // 2. Cálculo do total de dias completos e minutos restantes
        long diasCompleto = minutosTotais / (24 * 60);
        long minutosRestantes = minutosTotais % (24 * 60);

        BigDecimal valorBase = TETO_DIARIA.multiply(BigDecimal.valueOf(diasCompleto));

        if (minutosRestantes > 0) {
            BigDecimal valorPeriodoRestante;
            if (minutosRestantes <= 60) {
                valorPeriodoRestante = TARIFA_PRIMEIRA_HORA;
            } else {
                long minutosAposPrimeiraHora = minutosRestantes - 60;
                long horasAdicionais = (minutosAposPrimeiraHora + 59) / 60;
                valorPeriodoRestante = TARIFA_PRIMEIRA_HORA.add(
                    TARIFA_HORA_ADICIONAL.multiply(BigDecimal.valueOf(horasAdicionais))
                );
            }

            // Aplica teto da diária sobre o período restante
            if (valorPeriodoRestante.compareTo(TETO_DIARIA) > 0) {
                valorPeriodoRestante = TETO_DIARIA;
            }
            valorBase = valorBase.add(valorPeriodoRestante);
        }
    }
}
```

```
        // 3. Multiplicador pelo tipo de veículo
        BigDecimal valorComVeiculo =
valorBase.multiply(BigDecimal.valueOf(tipo.getFatorMultiplicador()));

        // 4. Desconto de convênio
        BigDecimal valorFinal = possuiConvenio
            ? valorComVeiculo.multiply(FATOR_DESCONTO_CONVENIO)
            : valorComVeiculo;

        return valorFinal.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```